

Braillix simulator - what it is, and what it should become

Written 2026-08-04. Live at <https://mridulnegi2005.github.io/Capstone/>

The long-term goal, in Mridul's words: **this project will stay on GitHub and help others understand the product, because of course they won't have the physical device.**

That single sentence should drive every decision below. The simulator is not a demo prop. **It is the documentation.** Anyone who finds this repo in three years has the web app and nothing else - no printed parts, no motor, no us to ask.

Where it stands today

Renders	one full cell - box, plates, cam disc, six linkages, six dots
Encoding	26 letters, digits 0-9 , capital / number / letter signs , 8 punctuation marks
Motion	shortest-path cam rotation, Step and Simulate, adjustable speed
Inspection	X-Ray toggle, orbit / pan / zoom, live decode panel
Honesty	every constant is parsed out of the CAD by <code>sim/extract_params.py</code> - the sim cannot drift from the design
Offline	Three.js is vendored. No CDN, no internet, no build step

That last row matters more than it looks. A simulator that quietly disagrees with the CAD is worse than no simulator, because people trust it.

The indicator system (added 2026-08-04)

The thing most people get wrong about braille: **indicators are cells too.**

There is no separate "capital A" pattern. There is a capital sign, and then an a. The mechanism has to index twice. On a one-cell device that means the reader feels two things in sequence, and the motor does two full moves.

Indicator	Dots	Meaning
Capital sign	6	the next letter is upper case
Number sign	3 4 5 6	digits follow, until a space
Letter sign	5 6	cancels number mode so a letter can follow a digit

Braille is lower case by default. Adding "small letters" required no new patterns at all - what was missing was the **capital** sign for the upper-case case. The old build displayed everything in caps, which was a display choice masquerading as an encoding.

Worked examples, all verified in `currentCells()`:

```
"a"      -> a          1 cell
"A"      -> ^ a        2 cells
"1"      -> # a        2 cells
"a1"     -> a # a      3 cells
"1a"     -> # a ~ a    4 cells  <- letter sign needed
"2.5"    -> # b . e    4 cells  <- . stays inside the number
"Braille" -> ^ b r a i l l e 8 cells
```

The cost is real and worth showing a panel: "A1" is four cam indexes for two characters. This is exactly why a real display has 20-40 cells and not one.

Roadmap

Ordered by **value per hour**, not by ambition. Each item says what it teaches.

Phase 1 - finish the single cell **(small, high value)**

- [] **Cell-count reality meter.** Show "12 characters -> 19 cells -> 19 motor moves -> 8.4 s".

Turns the indicator system from trivia into the argument for multi-cell.

- [] **Speed in real units.** The speed slider is unitless. Label it in ****characters per minute**** and compare with a human braille reading speed (~100-150 wpm). The gap is the honest limitation of the design and should not be hidden.

- [] **Chatter visualiser.** Highlight dots that move ***between*** letters without needing to.

CLUTCH_OPTIONS.md measured 5.4x wasted motion; shortest-path rotation already halved it. Show that, don't just claim it.

- [] **Guided tour mode.** A "start here" button that drives the camera and narrates 6-8 steps. Most visitors will not know what a cam is. Without this, orbit controls are a maze.

- [] **Text-to-braille panel.** Type a sentence, see the full cell-by-cell breakdown as a static chart, not just the animation.

Phase 2 - the electronics *(medium, high value for the panel)*

Mridul's ask: ***the brain pod, the electronics too should be visible.***

- [] **Model the ESP32 pod** - shell, lid, devkit, DC jack. The CAD already exists (esp32_pod_*.scad); this is an export-and-place job, not new design.

- [] **Wiring as geometry.** Draw the 13 Dupont connections as coloured tubes. Hover a wire, see which GPIO it is. This replaces a schematic nobody will open.

- [] **Live signal animation.** Show the four stepper coils energising in sequence during a move, and the hall sensor firing at home. **This is the piece that turns "a spinning model" into "I understand how it works".**

- [] **Component X-ray.** Extend the existing toggle so ULN2003, ESP32 and motor can each be isolated and labelled.

[!] Ordering note: do not model the electronics against the ***current*** stack until the R-07 hand-turn test settles whether the cam disc grows to dia 54. The pod is unaffected, so the pod can start now; the in-box layout should wait.

Phase 3 - multi-cell *(large, this is the "product" story)*

Mridul's ask: ***manually attaching more pods and see how it affects.***

- [] **Drag-to-attach cells.** Snap a second, third, fourth cell onto the pogo-pin edge.

The mechanical interface already exists (pogo_end_cap.scad, the ?X edge notch in mid_plate.scad).

- [] **Show what scales and what does not.** Each added cell is ****+1 motor, +1 driver, +1 I2C address**** but the ***reading speed per cell stays the same***. Adding cells buys you a longer line, not a faster one. A visitor should be able to discover that by dragging, which is far stronger than being told.

- [] **Power budget readout.** Cells x motor current vs the DC jack's rating. Find the wall live - that is a genuine engineering constraint the panel will ask about.

- [] **MCP23017 expander view.** Why the design moves to an I2C expander past N cells.

Phase 4 - outreach *(for the school for the blind)*

- [] **Audio layer.** Speak each character as it is displayed. The obvious point that gets

missed: **a sighted visitor learns from the screen, a blind visitor learns from the sound.** Right now the app is useless to the people it is built for.

- [] **Full keyboard control** and screen-reader labels on every button. Same reason.

- [] **Recorded MP4 walkthrough** as the offline fallback for a venue with no internet.

- [] **"Read this word" practice mode** - show a pattern, let the user guess the letter.

Phase 5 - repo as documentation *(cheap, do it last, matters most in year two)*

- [] **Landing page** in front of the simulator: what this is, who made it, what problem it solves, and a link to the CAD.

- [] **Explain the failures too.** The 2mm stack error, the cam pressure-angle problem, the arcs the printer could not run. **A repo that only shows the wins teaches nobody.**

- [] **BOM with live prices** and a "build one yourself" path.
 - [] **Print-your-own** links straight to the STLs.
-

Things deliberately NOT on this list

- **Physics simulation.** Rigid-body contact would be honest about jamming, but it is weeks of work and the answer comes cheaper from the hand-turn test on the real resin part.
 - **Photorealistic rendering.** Blender stills already exist. Prettier pixels teach nothing new.
 - **A rewrite in React/Three-Fiber.** The app is one HTML file and one JS file with no build step, and it will still open in 2031. That is a feature. Do not trade it for tooling.
-

The one rule

Every number in the simulator comes from the CAD, via `sim/extract_params.py`.

If a future feature needs a constant, add it to the extractor - never type it into `app.js`. The moment someone hard-codes a dimension, the simulator starts lying, and a lying simulator is worse than none at all.